

Coding & Solution

Starting Date	08 October 2026
Team ID	UNM1611PUNITHA
Project Name	AI STUDY BUDDY API

Instructions

Document the implemented solution, the development approach, major modules, and the code-quality practices actually present in the project. The content below is based on the supplied AI StudyBuddy API source code.

Solution Summary

Architecture: Node.js + Express REST API with MongoDB/Mongoose persistence and Google Gemini 2.5 Flash for AI generation.

Application entry: `index.js` loads environment variables, configures JSON parsing and CORS, creates the upload directory, mounts authentication/material/admin routes, exposes a health endpoint, connects to MongoDB, and starts the Express server.

Authentication: JWT access and refresh tokens are generated with separate secrets. Protected routes validate an access token from the `Authorization: Bearer` header; refresh uses the `x-refresh-token` header.

Authorization: middleware checks the authenticated user's role and restricts administrative routes to users with the `admin` role.

Study material flow: Multer accepts `.txt`, `.md`, and `.pdf` extensions with a 5 MB limit. The uploaded file is read into the Material document and the temporary disk file is deleted.

AI layer: a reusable Gemini helper sends prompts for summarization, flashcards, MCQ quizzes, and personalized study plans. Generated results are saved with the related material.

Implementation Modules

Layer	Implemented responsibility
Routes	auth.js, materials.js, admin.js define HTTP endpoints and route-level protection.
Controllers	authController.js, materialController.js and adminController.js implement application logic.
Middleware	auth.js validates JWTs and admin role; upload.js configures Multer storage, file filtering and 5 MB size limit.
Models	User and Material Mongoose schemas store accounts, study content, summaries, flashcards, quizzes and study plans.
Utilities	db.js connects to MongoDB; tokens.js creates JWTs; gemini.js wraps Gemini 2.5 Flash generation.

Code Quality Checklist

- Separation of concerns is present through routes, controllers, middleware, models and utility modules.
- Passwords are hashed with bcryptjs before persistence.
- JWT secrets and database/API configuration are read from environment variables.
- Protected resources check ownership for students; admins can access broader material/user operations.
- Uploaded files are filtered by extension and limited to 5 MB.
- Temporary uploaded files are deleted after their contents are stored.
- Global Express error handling returns a consistent JSON message and status code.
- Asynchronous controller errors are supported through express-async-errors.
- No automated test suite is implemented in the supplied package; the package test command is still the default placeholder.

Additional Notes / Comments

Important implementation accuracy: the README describes tokens as HTTP-only cookies, but the supplied source code currently returns tokens in JSON and uses Bearer/x-refresh-token headers. The documentation therefore records the source-code behavior rather than the README claim.

PDF handling limitation: the upload filter accepts .pdf files, but materialController.js reads every uploaded file using fs.readFileSync(path, 'utf-8'). A dedicated PDF text-extraction library is not present in the supplied project. PDF ingestion should therefore be validated before claiming reliable PDF content extraction.

Logout behavior: logout is stateless; the server returns a success message and does not revoke tokens.